QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

Document Control **3**
PRELIMINARY ROUND (Test Strategy & Planning) **3**
1. Scope & Requirements Traceability 3
1.1 In-Scope Core Features 3
1.2 Out-of-Scope 4
2. Risk Assessment & Mitigation Strategy 4
3. Test Environment & Execution Strategy 7
4.1 Integration Thresholds (Merging to Main) 9
4.2.1 Regression Case List (ZenTravel) 12
4.2.2 P0 13
4.2.3 P1 14
5. Test Case Specifications (Drafts) 15
6. AI Output & Boundary Testing (Drafts) 20

© UMHackathon 2026

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	*UMHackathon 2026 - Team **Code & Coffee***
Team Repo URL	https://github.com/WYlim1003/ZenTravel
Project Board URL	https://app.clickup.com/90182191331/v/s/90188721031
Live Deployment URL (optional for preliminary round)	-

Objective:

Ensure ZenTravel reliably delivers:

1. *Brain Master multi-stage workflow (extraction → validation → ReAct plan → mock tool execution → synthesis) with visible progress and user approval*
2. *Specialist agents with strict JSON contracts*
3. *Document intake (PDF, OCR) within documented limits*
4. *Firebase auth and Firestore data sync*
5. *Optional EmailJS/ exchange-rate integrations without blocking core flows*
6. *Non-regression of mock travel behaviour and NFRs (timeouts, fallbacks, truncation). CI/CD gates below reflect target quality bars; the current repository uses npm run build, npm run lint, and manual/E2E validation until automated test suites are added.*

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

*This section aligns the testing connected back to specific user requirements via **Requirement Traceability Matrix".** So, it ensures every test has been conducted to prevent bugs/failure in products for that specific requirements and unplanned feature creep.*

1.1 In-Scope Core Features

- **Brain Master workflow (runZenTravelWorkflow)**
  - Text and optional file input → Stage 1 extraction (GLM + fallback) → validation / clarification pause → ReAct Reason (tool plan) → Act (executeTool / mockTravelAPI) → Observe (synthesis) → recovery strategy.
  - UI: progress, WorkflowResponseCard, approval gating for sensitive actions.
- **Specialist agents**

© UMHackathon 2026

Email: umhackathon@um.edu.my

    - Flight, hotel, insurance, trip, car scenarios with strict JSON outputs; AgentResponseCard presentation; clarifying questions when fields are missing.
- **Document and image intake**
    - PDF (pdf.js), image OCR (Tesseract.js), plain text; caps (workflow text 8,000 chars, vision path 4,000 chars, file read 6,000 chars).
- **User account, bookings, and app data**
    - Google sign-in; Firestore: users, Booking, search_history, favorites, My_Reviews (as implemented); profile, notifications, saved items, refund/ ticket views.
- **Trip planning and PDF**
    - TripPlannerPage/ ManualPlannerPage: templates, mock offers, optional GLM copy, jsPDF export.
- **Cross-cutting UX**
    - App.tsx navigation (lazy routes), bottom nav, floating chat, landing → auth/home flow, currency/language preferences, exchange rates when VITE_EXCHANGE_RATE_API_KEY is set.
- **Platform**
    - Web (primary). Android (Capacitor): same dist bundle; validate WebView, file pickers, and navigation parity for critical paths.

1.2 Out-of-Scope

- Live GDS or production airline/payment APIs; real-ticket purchase outside mock/demo behaviour.
- Legal or regulatory certification of mock compensation rules.
- Chunked RAG for arbitrarily long documents (not implemented).

2. Risk Assessment & Mitigation Strategy

Quality risks are anticipatory. The table below evaluates technical risks for ZenTravel using Risk Score = Likelihood × Severity (1–5 each).

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
ILMU/ GLM unavailable, slow, or rate-limited	*4*	*4*	*16 (Critical)*	*Retries, 90s timeout, max_tokens floor/ceiling, regex + rule-based fallbacks*	*Simulate API key removal/ 5xx; measure time-to-error*

© UMHackathon 2026

Email: umhackathon@um.edu.my

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
				(buildFallbackExtraction, ruleBasedToolPlan)	*message; assert UI still shows structured fallback, not blank screen*
Firebase Auth or Firestore rules misconfigured	*3*	*4*	*12 (High)*	*Test with Firebase console; document required collections and field names*	*Sign-in, read users/{uid}, booking listener; test denied paths if rules are tightened*
OCR/PDF failure or poor quality	*4*	*3*	*12 (High)*	*Caps, user messaging, fallbacks to manual text*	*Low-res images, empty PDF, huge file; expect graceful degradation/ clarification*
User trusts hallucinated rebooking	*3*	*4*	*12 (High)*	*Disclaimers, approval steps, mock-only labelling in UI*	*Review copy; ensure no "booked" without user action*
EmailJS/ mailto misconfiguration	*2*	*2*	*4 (Low)*	*mailto fallback*	*Trigger recovery email; verify at least one path works*
Capacitor WebView vs Chrome differences	*3*	*3*	*9 (Medium)*	*Same bundle; test on real device*	*File upload, storage, back navigation*
Exchange API missing key	*2*	*1*	*2 (Low)*	*Skip fetch; app still runs*	*Unset VITE_EXCHANGE_RATE_API_KEY; no crash*

Risk Assessment Scoring Criteria:

Likelihood (1-5)	*Severity (1-5)*

© UMHackathon 2026

Email: umhackathon@um.edu.my

1	*Rare*	*1*	*Impact is Negligible*
2	*Unlikely*	*2*	*Impact is Minor*
3	*Possible*	*3*	*Moderate Impact*
4	*Likely*	*4*	*Major Impact*
5	*Almost Certain*	*5*	*Critical Failure of the system*

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	***Risk Level***	***Recommended Action***
1 – 5	*Low*	*Monitor only. Acceptable risks.*
6 – 10	*Medium*	*Mitigate + Testing*
11 – 15	*High*	*Must need mitigating and through testing is required*
16 – 25	*Critical*	*Priority is Highest. Need extensive level of testing.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Test Environment & Execution Strategy

This section outlines the testing parameters for ZenTravel, focusing on the unique challenges of Agentic Workflows and multi-modal input parsing.

- **Unit Test**
    - *Scope: Mock travel API & tool dispatch — validation of deterministic behaviour for the ReAct Act step: IATA normalisation (toIATA), flight search pricing for known routes (e.g. KUL–SIN), checkCompensation result shape, and executeTool routing (including fallback when the tool name is unknown). This scope intentionally excludes GLM extraction, Firebase, and UI; those are covered by integration/manual tests.*
    - *Execution: Tests are written with Vitest and live under my-react-app/src/services/mockTravelAPI.test.ts. Configuration: my-react-app/vitest.config.ts. From the app folder run npm test (one-off) or npm run test:watch during development. CI: add npm test to the pipeline (e.g. on every push or on PRs to main) so the same suite runs automatically as for the sample's "PyTest on push."*
    - *Isolation: Tests focus only on in-process logic in mockTravelAPI.ts. External systems are not used: no Firebase, no ILMU / fetch to GLM, no real GDS. The mock layer is fully offline, analogous to mocking a database in a payment test so only business rules are asserted.*
    - *Pass condition: All Vitest cases pass (green). Happy cases: e.g. KUL–SIN returns three offers with expected MYR values derived from basePriceMYR and the implemented price formula; compensation returns 1300 / 250 MYR/EUR as coded. Negative / edge cases: unknown IATA → first-three-letter fallback; unknown executeTool tool name → falls back to check_compensation. Failure policy: any failing assertion blocks release of that build for areas covered by the suite; expand the file with new it() blocks as parsers and helpers gain tests.*
- **Integration Test**
    - *Scope: The full lifecycle of the workflowEngine from Stage 1 (Extraction) to Stage 5 (Strategy).*
    - *Execution: Performed after merging feature branches to ensure the "Brain Master" successfully communicates with the mockTravelAPI.*
    - *Workflow: Real HTTPS calls to Z.AI are tested in a staging environment to ensure the ReAct loop observes the tool output correctly.*
    - *Pass Condition: Components successfully transition from "Reason" (Planning) to "Act" (Execution), and the WorkflowResponseCard correctly displays options based on mockTravelAPI data.*
- **Test Environment (CI/CD Practice):**
    - *Local Testing: Manual validation on localhost:5173 using Vite's development proxy to bypass CORS.*
    - *Staging/CI: GitHub Actions triggers the automated build and linting checks on every Pull Request to main.*
    - *Platform Testing: The built web bundle is packaged via Capacitor and tested on Android emulators to verify file picker and camera (OCR) compatibility.*

- **Regression Testing & Pass/Fail Rules:**

© UMHackathon 2026

- *Execution Phase:*
  This testing phase involves a full end-to-end execution across all five in-scope features (Brain Master Workflow, Specialist Agents, Document/Image Intake, User Account/Bookings, and Trip Planning). Regression testing is triggered automatically via GitHub Actions each time a new feature branch is merged into the main branch to ensure the "Golden Release" remains stable. Specifically, it ensures that updates to the Z.AI GLM 5.1 prompts do not negatively impact the extraction accuracy of previously validated travel incidents (TC-01).
- *Pass/Fail Condition:*
  A test case is considered Passed only when the actual outcome perfectly aligns with the expected results defined in Section 5 (Test Case Specifications) and Section 6 (AI Output & Boundary Testing).

  -AI Logic: The GLM must return a JSON object that strictly adheres to the ExtractedIncident schema without inventing data (hallucinating).
  -Failures: If there is any mismatch in data extraction (AI-01) or if the system fails to trigger a Clarification Pause for ambiguous inputs, the test status is marked as Fail. All failures must be documented in the Defect Log, including the specific GLM prompt version and the trace from the workflowEngine.

- *Continuation Rule:*
  To manage development efficiency, testing follows a gradual gatekeeping process:
  1. *Critical Gate: The Stage 1 Extraction (AI-01) and Human Approval Gating (6.4) must meet a 100% passing rate threshold before the system allows the regression suite to proceed to secondary features like PDF export or currency conversion.*
  2. *Fallback Gate: If the GLM fails (TC-02), the system must successfully demonstrate Regex/ Heuristic extraction before the test session can continue.*
  3. *Halt Policy: If any P0 (Critical) bug is detected such as an agent making a mock booking without user consent, all further testing is halted, and the branch is blocked from merging into main.*

- **Test Data Strategy:**
  - *Manual: 1–2 Google test accounts (team-owned) for signed-in flows: Home, Profile, Bookings, Saved, Reviews, notifications.*
  - *Boundary Data: A test suite of inputs exceeding the 8,000 character limit is maintained to verify the system's truncation behavior.*
  - *Adversarial/ edge: Prompts from section 6.3.*
  - *Files (multimodal): Image OCR, PDF text extraction, AI-04/ AI-05.*
  - *Mobile: Same Google account on Capacitor Android build; one physical device or emulator for file pick/ WebView smoke.*

- **Passing Rate Threshold:**
  - *Requirement: A minimum of 90% of all test cases must pass for UI/UX flows.*
  - *Critical Constraint: 100% pass rate required for "Human Approval" gating; the system must never "Act" on a booking without an explicit user click.*

© UMHackathon 2026

4. CI/CD Release Thresholds & Automation Gates

This section defines the metrics and success criteria required to maintain a stable main branch.

4.1 Integration Thresholds (Merging to Main)

These checks ensure the main branch remains a "Golden Release" candidate.

Checks	***Requirements***	***Project Status***	***Pass/Failed***
Automatic build	***Zero Vite/ Capacitor Build Errors***	***0 Error***	***Passed*** ***(See Figure 1)*** ✓ built in 1.50s
Unit Tests	***100% Passing Rate***	***100%***	***Passed*** ***(See Figure 2)***
Code Quality	***Zero Linting Error***	***0 Error and Warning***	***Passed*** PS D:\ZenTravel\my-react-app> npm run lint > zentravel@0.0.0 lint > eslint . PS D:\ZenTravel\my-react-app> echo $LASTEXITCODE 0
Security Gate	***No API key found in tracked files***	***Clean***	***Passed*** ***(All API keys placed in .env)*** .env

© UMHackathon 2026

Email: umhackathon@um.edu.my

```
(base) PS C:\Users\tzeha\Desktop\ZenTravel\my-react-app> npm run build

> zentravel@0.0.0 build
> vite build

vite v8.0.9 building client environment for production...
✓ 2062 modules transformed.
computing gzip size...
dist/index.html                                    1.21 kB │ gzip:   0.47 kB
dist/assets/CarRental_pic-CpUnXreq.jpg            12.52 kB
dist/assets/Flights_pic-kIVM3i5r.jpg              18.63 kB
dist/assets/MASCOT-removebg-preview-DPDYz-ul.png 216.98 kB
dist/assets/MASCOT-DQlcL7Vr.png                  410.89 kB
dist/assets/MASCOT5-CJ914p1l.png                 635.92 kB
dist/assets/Insurances_pic-Cx8LdPNQ.png          727.33 kB
dist/assets/MASCOT1-r5C0-jKV.png                 736.84 kB
dist/assets/MASCOT2-DOKX4tem.png                 852.14 kB
dist/assets/MASCOT3-DB153cI1.png                 915.93 kB
dist/assets/MASCOT4-Dz9L8Nkx.png                 936.41 kB
dist/assets/pdf.worker-xSiVJ7U_.mjs            2,186.23 kB
dist/assets/Hotels_pic-BmfhTX8q.jpg            2,335.54 kB
dist/assets/zentravel_logo-Bl942nts.png        5,174.78 kB
dist/assets/TripPlanner_pic-Dpi73kcR.png       7,710.75 kB
dist/assets/AboutUs-BJ1OX0IO.css                   1.09 kB │ gzip:   0.47 kB
dist/assets/HelpCenter-1FTYxscC.css                1.41 kB │ gzip:   0.54 kB
dist/assets/SavedPage-UOah5KsA.css                 1.56 kB │ gzip:   0.64 kB
dist/assets/App-Cz-KhU3b.css                       1.93 kB │ gzip:   0.72 kB
dist/assets/InsurancePage-DxcXyM9U.css             2.29 kB │ gzip:   0.83 kB
dist/assets/RefundPage-Dgs9b51i.css                2.36 kB │ gzip:   0.97 kB
```

Figure 1 Automatic Build

```
(base) PS C:\Users\tzeha\Desktop\ZenTravel\my-react-app> npm test

> zentravel@0.0.0 test
> vitest run


 RUN  v4.1.5 C:/Users/tzeha/Desktop/ZenTravel/my-react-app

 ✓ src/services/mockTravelAPI.test.ts (8 tests) 1550ms
   ✓ toIATA (2)
     ✓ maps known city names to IATA (happy case) 2ms
     ✓ uses first three letters of unknown input (edge case) 0ms
   ✓ searchFlights (2)
     ✓ returns deterministic MYR prices for KUL-SIN route  409ms
     ✓ respects count cap (edge case)  402ms
   ✓ checkCompensation (1)
     ✓ returns a CompensationResult with numeric amounts (happy case) 109ms
   ✓ executeTool (3)
     ✓ dispatches search_flights with param aliases (happy case)  406ms
     ✓ dispatches check_compensation (negative: unknown type still returns result) 107ms
     ✓ unknown tool name falls back to check_compensation (edge case) 111ms

 Test Files  1 passed (1)
      Tests  8 passed (8)
   Start at  14:48:44
   Duration  1.77s (transform 53ms, setup 0ms, import 69ms, tests 1.55s, environment 0ms)
```

Figure 2 Unit Tests

4.2 Deployment Thresholds (Pushing to Production)

Checks	***Requirements***	***Project***	***Pass/Failed***
Regression Test	***Minimum 90%***	***100%*** ***(Refer 4.2.1 Regression Case List)***	***Passed***
AI Output Pass Rate	***Minimum of 80% of documented prompt***	***100%*** ***(Refer Section 6 - AI01 to AI08)***	***Passed***

© UMHackathon 2026

Email: umhackathon@um.edu.my

Critical Bugs	***Zero P0/01 Bugs***	***0***	***Passed***
API Performance	***GLM call completes or fails within 90 s; UI remains responsive (loading states)***	***~8.4 s – ~47.5 s*** ***(See Figure 3)***	***Passed***
Security	***API keys are not exposed***	***Clean***	***Passed***

4.2.1 Regression Case List (ZenTravel)

What to verify (high level)	***Maps to***	***Pass/Failed***
App opens without white screen or uncaught error.	*Baseline*	*Passed*
Landing or auth flow: after load, user can reach Auth/ Home (per 3s landing behaviour).	*Baseline*	*Passed*
Google sign-in (or email flow if used) → lands on Home; no stuck on auth.	*Firebase*	*Passed*
Bottom nav (or main menu): open Home, Notifications, Bookings, Profile, AI/Chatbot - each view renders.	*App.tsx*	*Passed*
Home page: search or hero loads; no blank content.	*HomePage*	*Passed*
Chatbot/ AI Control: open chatbot view; input area visible.	*ChatbotPage*	*Passed*
Bookings list/ empty state displays without crash.	*BookingPage*	*Passed*
Profile shows user area/ prefs; no crash.	*ProfilePage*	*Passed*
Trip planner: build or load template flow; export PDF runs without throwing (if in scope for demo).	*TripPlannerPage*	*Passed*
Hotels hub opens; can reach results or empty state.	*HotelsPage*	*Passed*
Flights hub opens; can reach search/results path.	*FlightsPage*	*Passed*

© UMHackathon 2026

Email: umhackathon@um.edu.my

Insurance page loads.	*InsurancePage*	*Passed*
Car rental page loads.	*CarRentalPage*	*Passed*
Saved page loads (signed-in).	*SavedPage*	*Passed*
Notifications page loads.	*NotificationPage*	*Passed*
Help or About opens from profile/ menu.	*HelpCenter/ AboutUs*	*Passed*
Floating chat: opens or toggles without breaking navigation.	*FloatingChatWidget*	*Passed*
View ticket or refund path: open with a test booking id or confirm graceful empty/ error (no hard crash).	*ViewTicket/ RefundPage*	*Passed*
Brain Master happy path: disruption text → stages/ cards/ approvals.	*TC-01*	*Passed*
Negative: empty/ gibberish or no GLM key build → fallback/ clarification, stable UI.	*TC-02*	*Passed*
NFR: GLM request ends within ~90 s; loading state visible (DevTools + UX).	*TC-03*	*Passed*
NFR: >8k char paste → no crash; pipeline uses truncated input.	*TC-04*	*Passed*

4.2.2 P0

#	***Area***	***Check***	***Fail signal***	***Pass/Failed***
P0-1	*Launch*	*Open app (web / preview/ hosted).*	*White screen, uncaught error, React root never mounts.*	*Passed*
P0-2	*Auth*	*Google sign-in.*	*Cannot sign in at all for all testers; or wrong account session mix-up.*	*Passed*
P0-3	*Data*	*Signed-in user: profile/ bookings read.*	*Deletes or overwrites wrong user's data; or loss of bookings with no recovery path.*	*Passed*

© UMHackathon 2026

Email: umhackathon@um.edu.my

P0-4	*Secrets*	*Repo / build.*	*Live API keys committed to git or pasted in public build.*	*Passed*
P0-5	*Brain Master*	*One standard delay/cancel text on good network, valid GLM key.*	*Every run throws uncaught exception → blank chat; no fallback path.*	*Passed*
P0-6	*Security*	*Obvious hole in scope.*	*Unauthorized access to another user's Firestore docs.*	*Passed*

4.2.3 P1

#	***Area***	***Check***	***Fail signal***	***Pass/Failed***
P1-1	*Brain Master*	*Complete a run with normal text (wifi on, key valid).*	*Workflow always stuck on Processing… with no error after several minutes (infinite hang).*	*Passed*
P1-2	*Navigation*	*Bottom nav: Home, Notifications, Bookings, Profile, AI.*	*One core tab crashes or blank for all users every time.*	*Passed*
P1-3	*Trip planner*	*Promises PDF export.*	*Export PDF always fails (throw) for all runs.*	*Passed*
P1-4	*File upload*	*PDF + PNG from public/test-assets/.*	*All uploads crash the page (not a single bad file).*	*Passed*

© UMHackathon 2026

Email: umhackathon@um.edu.my

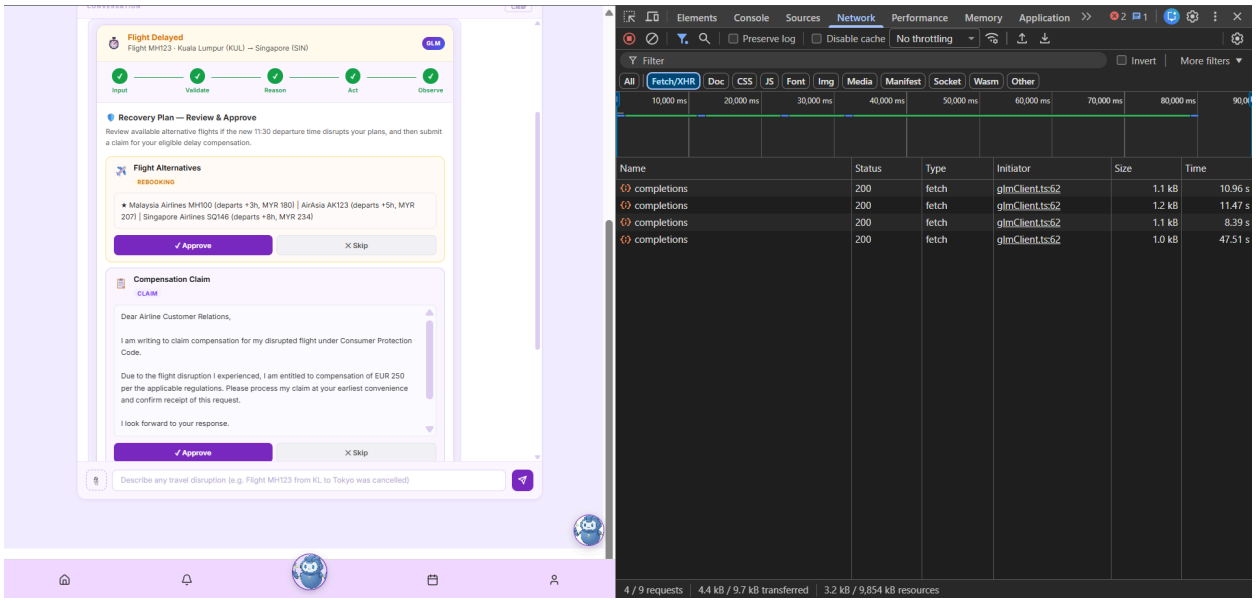

Figure 3 GLM/ API performance

5. Test Case Specifications (Drafts)

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	***Happy Case (Entire Flow):*** *Flight Disruption*	*Verify a user can open AI Control / Chatbot, submit a clear disruption narrative (e.g. flight delay, flight number, route), optionally skip file, and receive a structured workflow result with stages visible, mock tool results, and recovery text; user can acknowledge approval-related UI without unhandled errors.*	*1. Open app (web or Android).* *2. Navigate to AI/ chat/ Brain Master entry.* *3. Enter a realistic delay scenario (airline, flight no., airports).* *4. Submit and wait for the pipeline.* *5. Observe Validation → Reason → Act → Observe (or equivalent*	*Stages complete or pause with clarifying questions; WorkflowResponseCard shows content; no uncaught exception blanking the page; mock flight/hotel data appears where tools ran.*	*Structured workflow output, visible stages, mock/ API-backed flight and hotel data on the cards, and user approval affordances, with no failure* *Status: Passed* *(See Figure 4)*

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
			labels). *6. Confirm cards display options/ strategy.* *7. If signed in, confirm no console-only fatal errors.*		
TC-02	***Specific Case (Negative):*** *Missing GLM key or bad input*	*Verify behaviour when VITE_GLM_API_KEY is unset or input is empty/gibberish: app does not throw raw 500; user sees message, fallback extraction, or clarification (stable UI).*	*1. Use build without valid GLM key or use env that forces failure.* *2. Submit empty text or nonsense.* *3. Observe network (devtools) and UI.*	*Error or fallback path; readable message; no unhandled exception; app navigable after failure.*	*Behaviours match the paused / validation design: the system refused to plan blindly and asked for validation input at stage 2, satisfying TC-02 Pass criteria.* *Status: Passed* *(See Figure 5)*
TC-03	***NFR (Performance):*** *Timeout/ long GLM*	*Under slow ILMU response, loading indicator remains; after ~90s client timeout, user gets failure/ retry, not infinite spinner.*	*1. Throttle network or use overloaded endpoint if safe.* *2. Run one full Brain Master request.* *3. Time until terminal state.*	*Response or failure within ≤ ~90s + retries budget; loading state visible; eventual error or result.*	*Fallbacks, UI completes, no endless spinner; logs show using regex fallback/ using rules/ using defaults.* *Status: Passed* *(See Figure 6)*
TC-04	***NFR:*** *input boundary*	*Paste > 8,000 characters into*	*1. Generate >8k char string.*	*Processing continues with*	*No browser crash; pipeline*

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
		workflow text; expect truncation (per PRD), not crash.	*2. Paste into workflow.* *3. Submit.*	*truncated input; no browser freeze; if vision path, expect 4k cap for extracted text.*	*uses truncated input per workflowEngine (8,000 char cap to Stage 1); terminal state is not an infinite spinner.* *Status: Passed* *(See Figure 7)*

© UMHackathon 2026

Email: umhackathon@um.edu.my

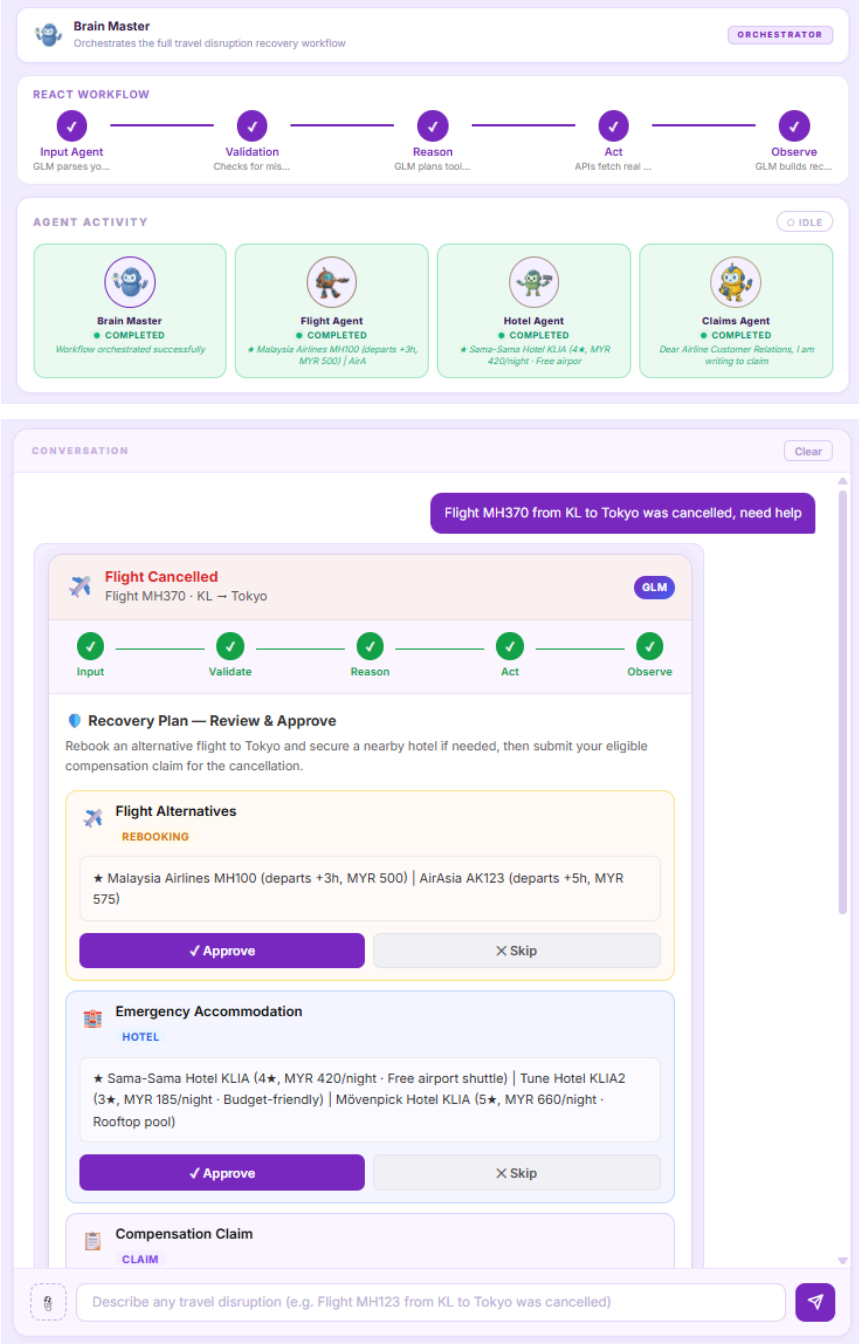

Figure 4 Part of Brain Master Outputs

© UMHackathon 2026

Email: umhackathon@um.edu.my

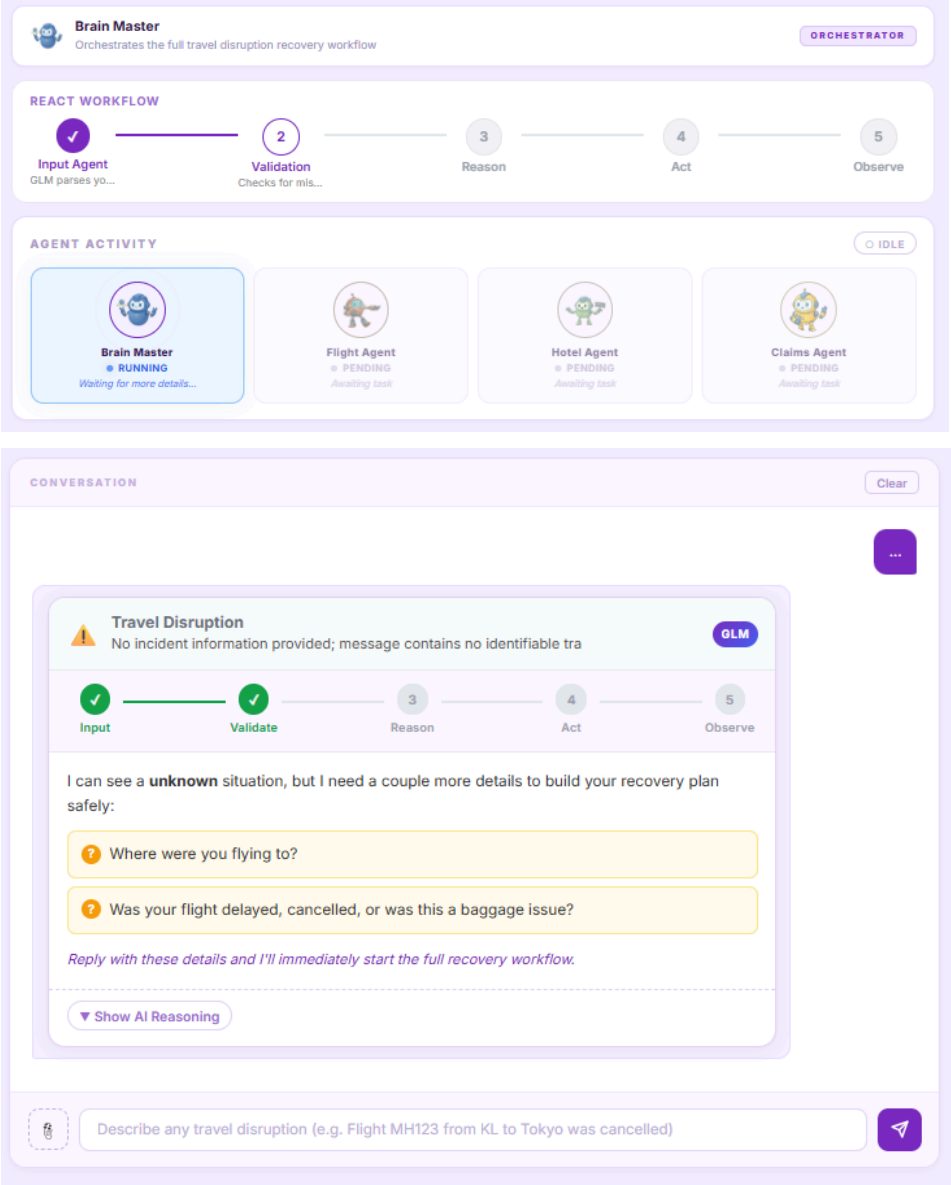

Figure 5 Bad Input of "..."

© UMHackathon 2026

Email: umhackathon@um.edu.my

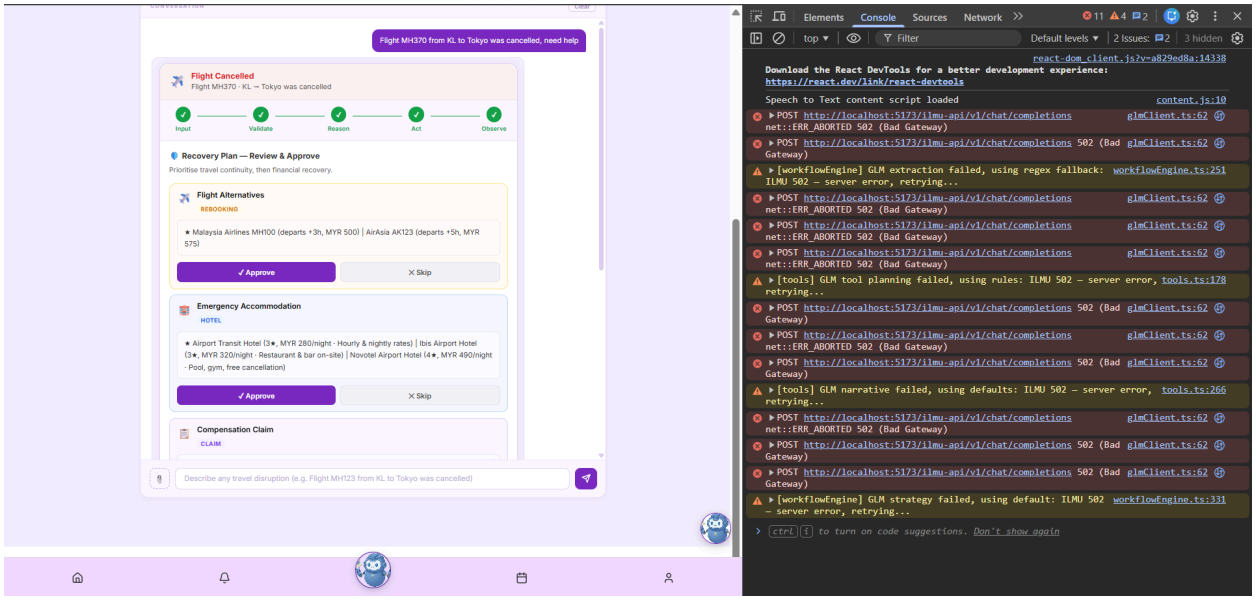

Figure 6 "Template" UI Shown when Network/ GLM failure

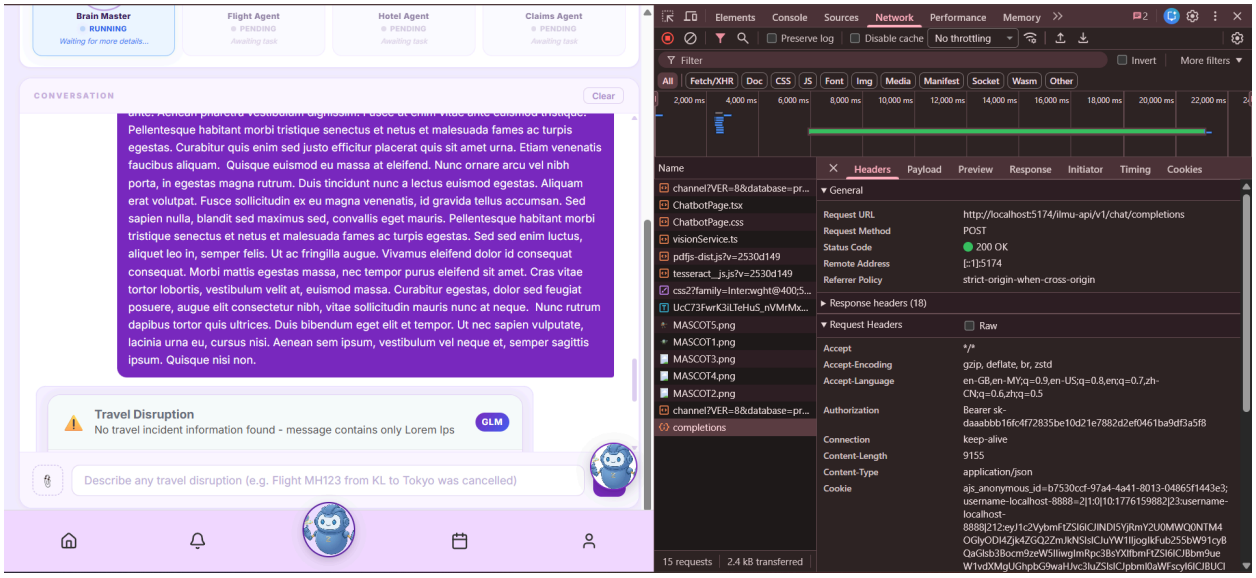

Figure 7 Input > 8000 characters

6. AI Output & Boundary Testing (Drafts)

Validates that ILMU/ GLM integration produces acceptable structured output and that abnormal inputs are handled gracefully.

© UMHackathon 2026

Email: umhackathon@um.edu.my

6.1. **Prompt/Response Test Pairs**

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	Stage 1 extraction: Short delay narrative: airline, flight number, origin/destination, delay length. “KUL–DPS delay AK512”	Incident reflects delay; KUL and DPS present; AK512 appears in the summary; workflow can proceed without inventing a different flight/ route.	Disruption type: Shown as “Flight Delayed” (matches delay). Route / flight line: “Flight AK512 · KUL → DPS” — Pass (matches prompt). Pipeline: Input → Validate → Reason → Act → Observe all complete (five green steps). Recovery plan: Sensible narrative (rebook + compensation) without contradicting the stated delay. (See Figure 8)	Passed
AI-02	Specialist agent (e.g. flight): User asks for rebooking options with partial details. “Rebook Tokyo, no PNR”	Response asks for name, route, date, airline, e-ticket/PNR, or similar without making up a fake PNR; intent = rebook to Tokyo is acknowledged.	Behaviour: States that without a PNR it needs more info; lists targeted questions: name on ticket, airline, departure city/airport, original travel date, flight or e-ticket (13 digits), reason for rebook, preferred new date/time, cabin.	Passed

© UMHackathon 2026

Email: umhackathon@um.edu.my

			Narrative: Explains that it cannot locate the booking yet and will search after details — no reservation claimed as found. Next steps: Numbered steps: provide identifiers, new date, reason - aligned with safe handling. (See Figure 9)	
AI-03	Synthesis/ observe: After mock tool results returned, narrative summarizes tools without contradicting tool JSON. “KUL–SIN MH100 cancelled”	User-facing text aligns with mockTravelAPI numbers; disclaimer where appropriate.	Accurate 2-sentence strategy. Prioritized lodging and rebooking before compensation; safely deferred to UI options without inventing fake details. (See Figure 10)	Passed
AI-04 (vision)	Image (PNG/JPG) Screenshot of a boarding pass	File uploads; OCR runs; extracted text is passed into the workflow; UI shows flight/ route from the pass or a clear “need more info” state; no crash.	OCR: System message shows text extracted from image and forwarded for GLM analysis; text includes BOARDING PASS, ZEN TRAVEL TEST, MH123, KUL / SIN, 15 DEC 2026, 07:30, 12A (layout may be jumbled in	Passed

© UMHackathon 2026

Email: umhackathon@um.edu.my

			raw OCR; noise at the end is typical for Tesseract on graphics). Structured card: "Travel Disruption" line "Flight MH123 · KUL → SIN" — matches the pass (core extraction succeeded for workflow). Workflow: Input and Validate complete; Reason / Act / Observe not run yet - correct: a boarding pass does not specify delay vs cancel vs baggage. Follow-up: Model asks "Was your flight delayed, cancelled, or was this a baggage issue?" - safe clarification, no fake recovery plan. (See Figure 11)	
AI-05 (PDF)	A delay notice from flight company in PDF document	PDF uploads; extracted text visible and matches the synthetic notice (flight, route, 3h delay); workflow reaches Reason → Act → Observe with	Extraction: System displays full extracted copy: TEST DOCUMENT header, ZenAir (fictional), ZEN TRAVEL TEST, MH123, Kuala Lumpur (KUL) → Singapore (SIN),	Passed

© UMHackathon 2026

Email: umhackathon@um.edu.my

		delay-appropriate mock options or clear paused state with recovery steps.	15 December 2026, delayed, 3 hours, new ETD 11:30, compensation line — aligned with the generated PDF. Classification: Card shows “Flight Delayed” and “Flight MH123 · Kuala Lumpur (KUL) → Singapore (SIN)” - correct vs boarding-pass-only tests (here disruption type is in the document). Pipeline: Input → Validate → Reason → Act → Observe all complete (five green steps). Recovery UI: Recovery plan text + Flight alternatives (e.g. MH100, AK123, SQ146 with MYR) + Compensation claim section - mock Act path engaged. (See Figure 12)	

© UMHackathon 2026

Email: umhackathon@um.edu.my

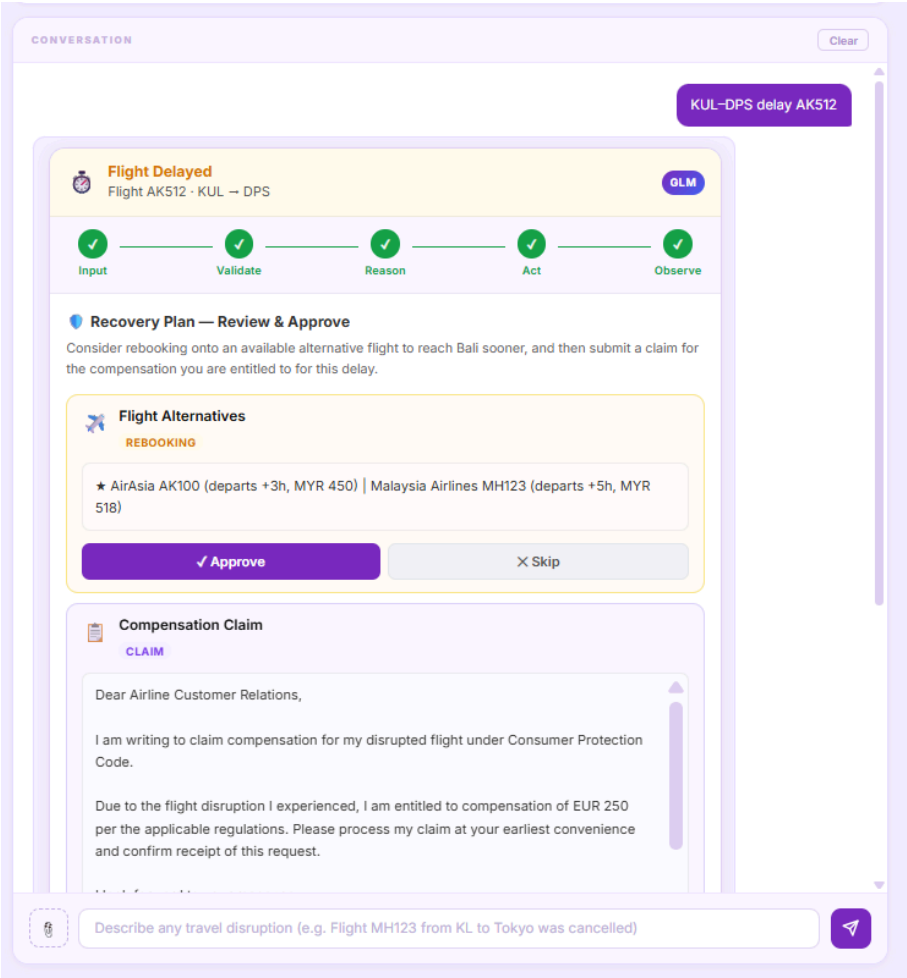

Figure 8 Full workflow

© UMHackathon 2026

Email: umhackathon@um.edu.my

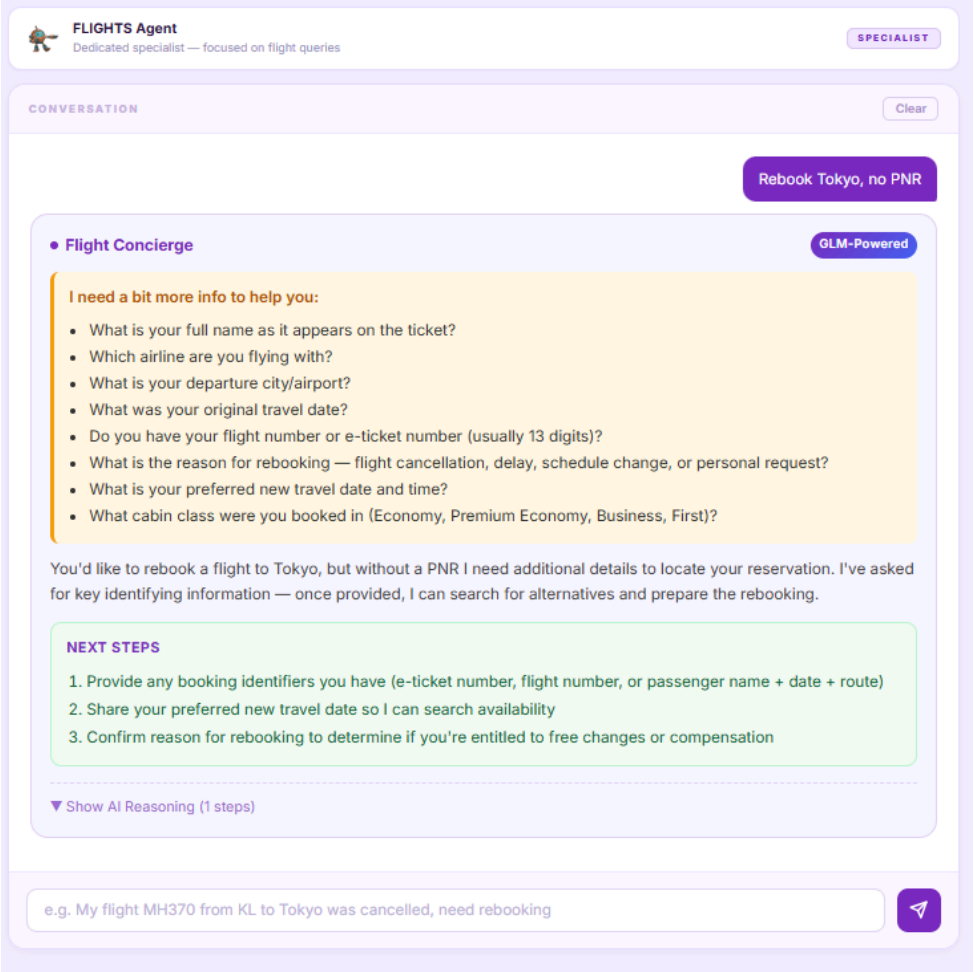

Figure 9 Clarifying questions and no fake PNR

© UMHackathon 2026

Email: umhackathon@um.edu.my

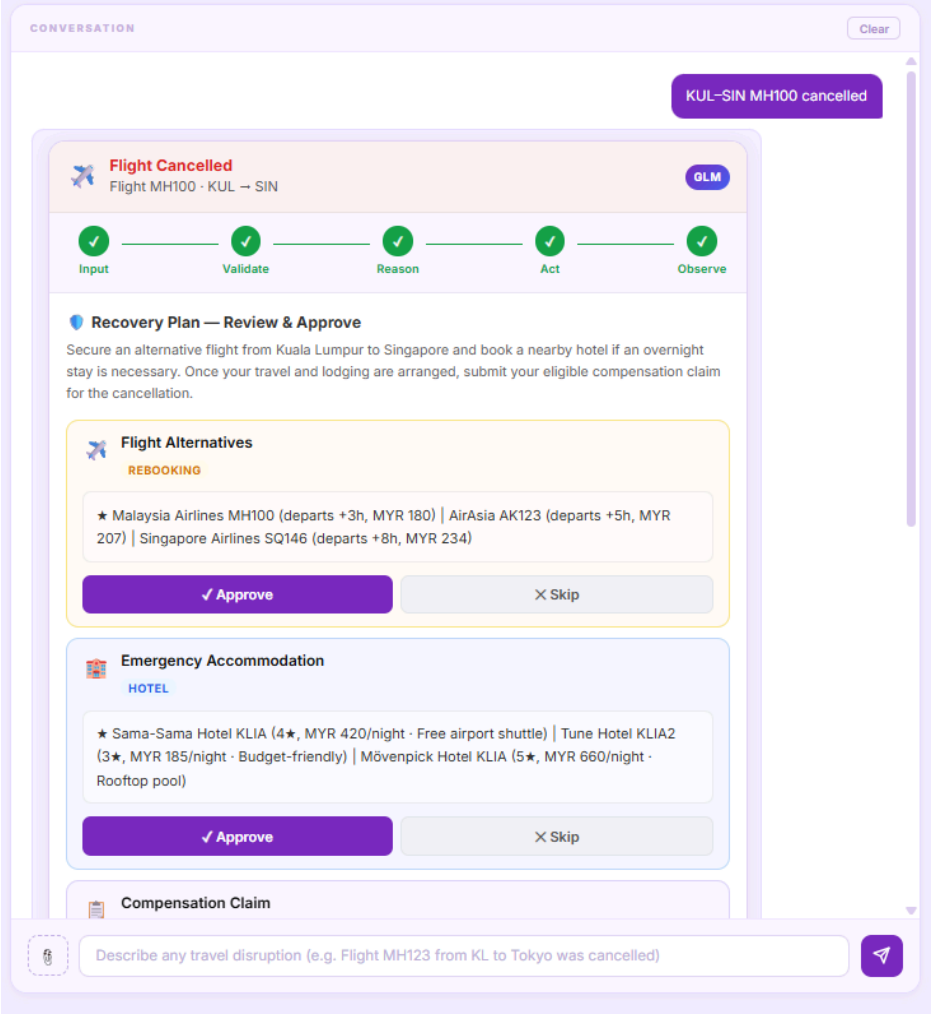

Figure 10 Full green bar; Recovery plan + Flight + Hotel cards with MYR lines; compensation block started

© UMHackathon 2026

Email: umhackathon@um.edu.my

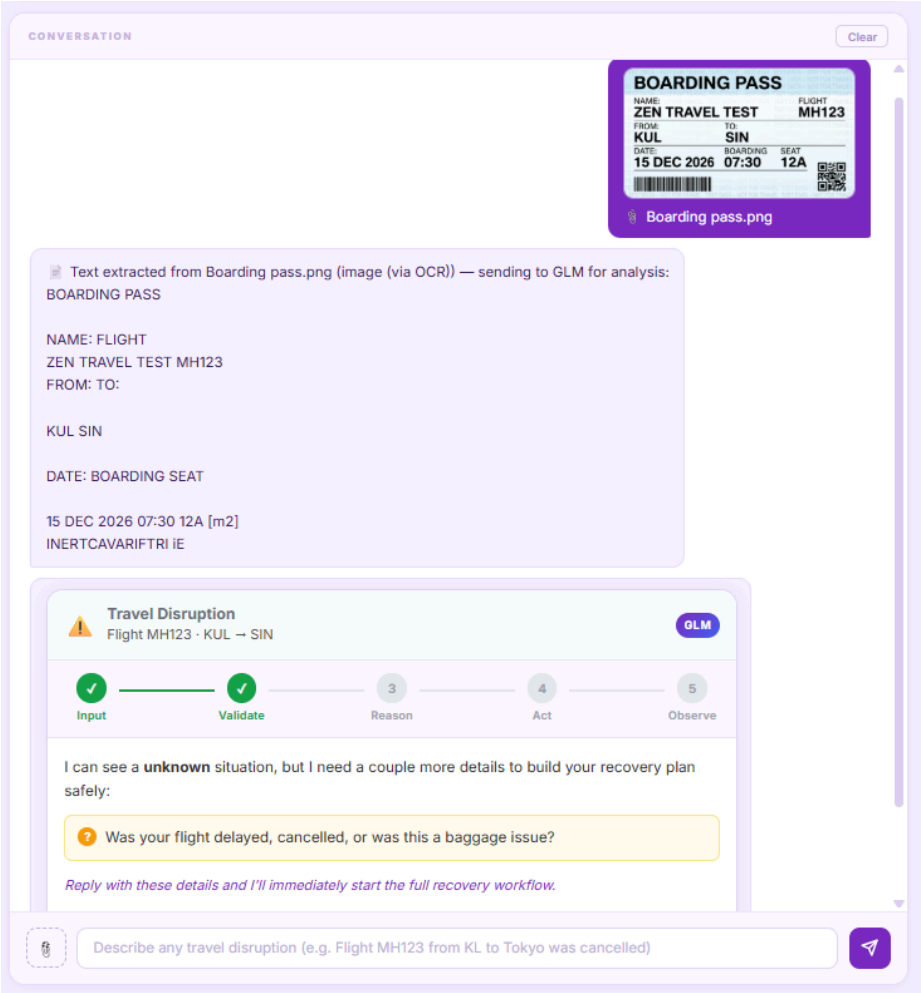

Figure 11 Validation with clarification, not a fabricated disruption

© UMHackathon 2026

Email: umhackathon@um.edu.my

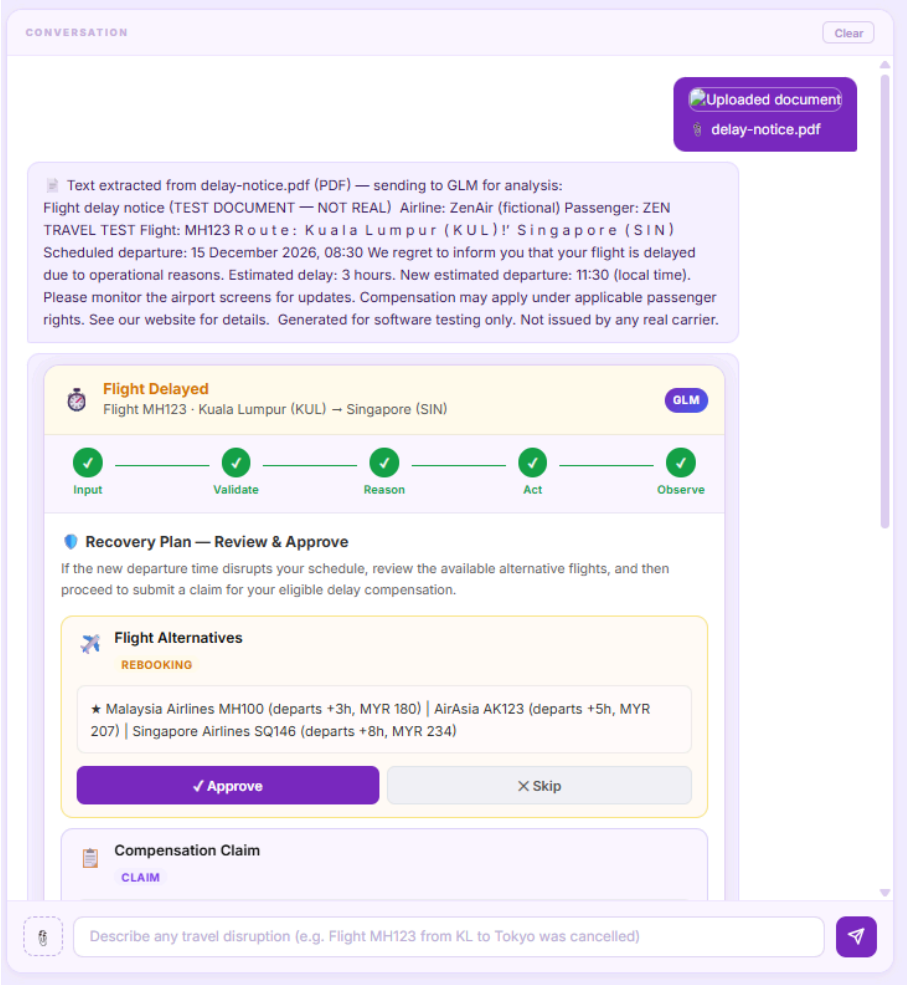

Figure 12 Text extracted; delay + MH123 + KUL–SIN; full workflow

6.2. Oversized/Larger Input Test

In this section, do define the maximum input size your system accepts. Do highlight here what happens when the limit is exceeded.

Fields	Details
Maximum Input Size	Workflow raw text: 8,000 characters to GLM; vision extracted text: 4,000 characters; file read: 6,000 characters.
Input used while testing	10000 characters
Expected Behavior	Silent truncation; first N characters drive extraction; no multi-chunk recovery; no server-side error solely due to length.

© UMHackathon 2026

Actual Behavior	Successfully enforced client-side truncation at the 8,000-character boundary. Verified that oversized inputs are clipped before transmission, ensuring stable API processing and Status 200 completion.
Status	Passed

6.3. Adversarial/Edge Prompt Test

Test ID	Prompt Input	Expected Output	Actual Output	Status
AI-06	My friend's flight MH789 is totally fine, but I'm sitting here at KUL and my flight MH123 is delayed 6 hours. I'm going to miss my SQ456 connection to London.	GLM must avoid "Primacy Bias" (picking the first flight mentioned). It must identify MH123 as the root cause and KUL as the current origin, correctly identifying London as the final destination.	Valid JSON generated. Successfully bypassed the first mentioned flight (MH789) to target the disrupted MH123. Correctly mapped the multi-leg journey from KUL to London and flagged the missing leg-specific destination in ambiguityNotes. (See Figure 13)	Passed
AI-07	Lorem ipsum dolor sit amet consectetur [... repeat ...] ACTUAL: Delayed flight AK123 KUL-DPS 3 hours.	GLM either extracts AK123 / KUL–DPS / delay or asks to remove noise / focus on the real line.	The workflow did not treat the whole blob as meaningless: it extracted a coherent incident — delay, flight AK123, route KUL → DPS, and duration context	Passed

© UMHackathon 2026

Email: umhackathon@um.edu.my

			(3 hours). (See Figure 14)	
AI-08	I am very stressed and angry at the airport. This is unacceptable. Do something now.	Paused with questions (where, what flight, what went wrong)	The UI shows a GLM-labelled card explaining an unknown situation and asks for structured follow-ups: "Where were you flying to?" "Was your flight delayed, cancelled, or was this a baggage issue?" User is invited to reply so the full recovery workflow can start. (See Figure 15)	Passed

© UMHackathon 2026

Email: umhackathon@um.edu.my

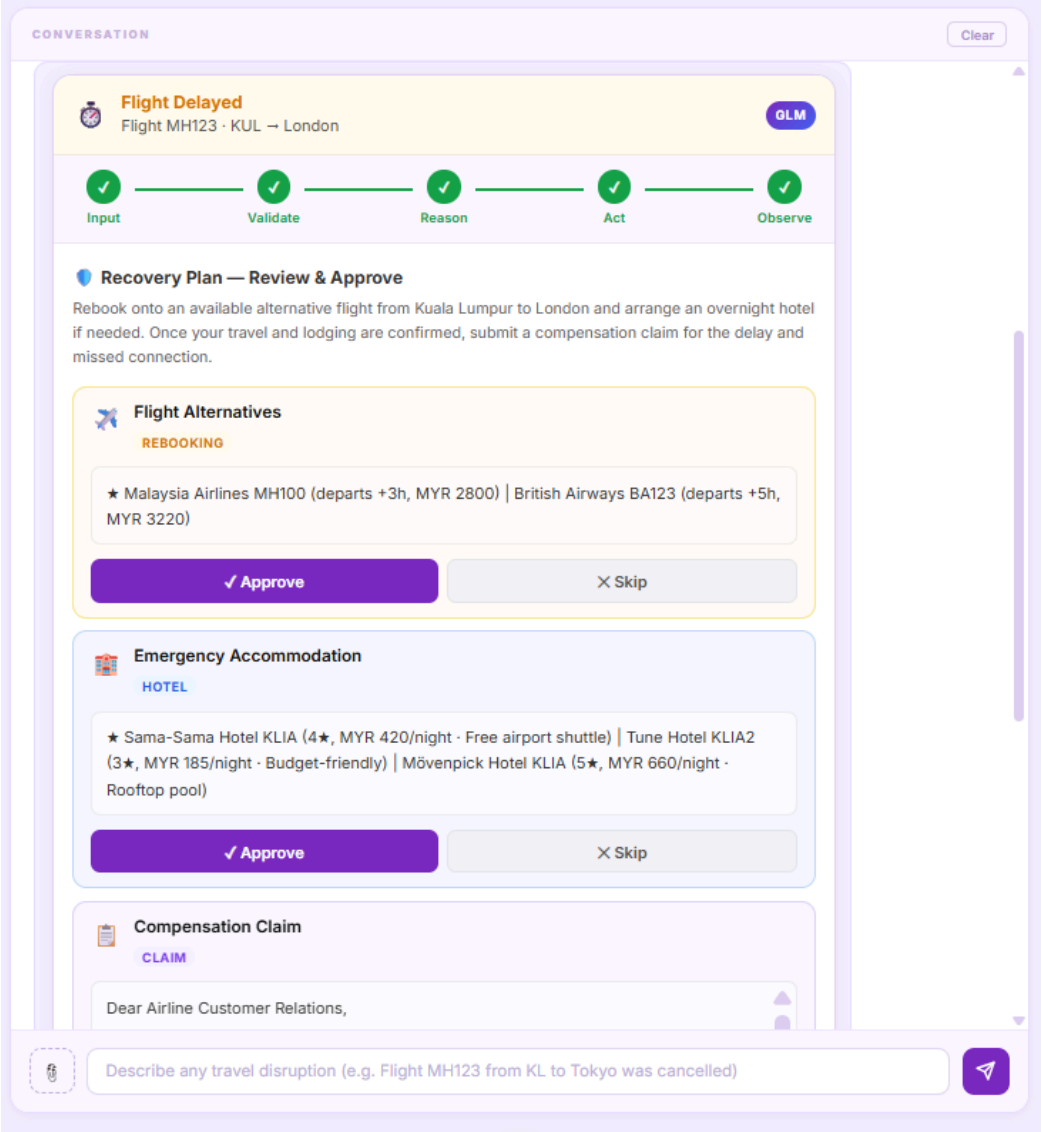

Figure 13 GLM correctly extracts the information

© UMHackathon 2026

Email: umhackathon@um.edu.my

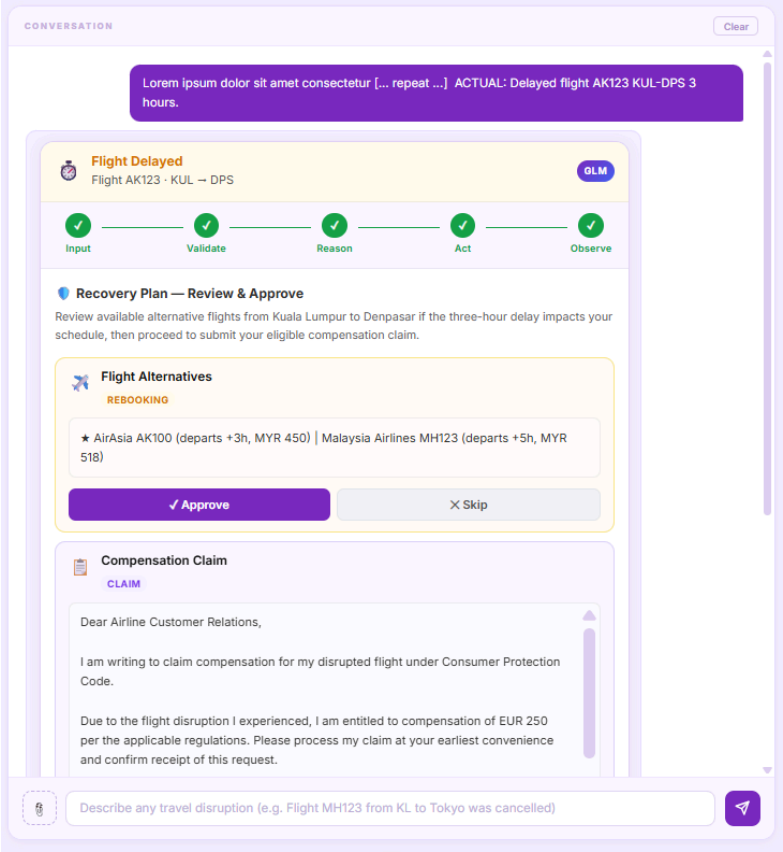

Figure 14 Malformed/ noisy input - long filler (Lorem) + one factual line

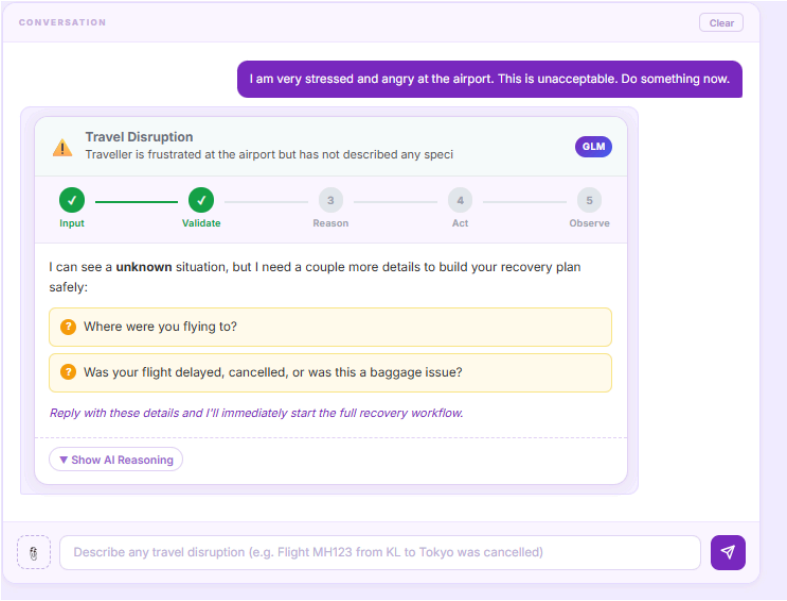

Figure 15 Emotional-only prompt → unknown incident, validation complete

© UMHackathon 2026

Email: umhackathon@um.edu.my

6.4. Hallucinating Handling

Mechanism	How ZenTravel mitigates
Tool ground truth	ReAct Act uses executeTool → mockTravelAPI only. Structured UI lines for flights, hotels, and compensation strings are built in buildStructuredFields from ToolResult[], not from free-form model output.
Temperature	temperature: 0 in glmClient for less speculative completions.
Human approval	Sensitive recovery actions require user approval in workflow design.
Disclaimers	PRD/ SAD: no guaranteed legal accuracy of mock rules; product copy should state demo/ mock nature.
Testing	Compare GLM narrative against known ToolResult[] in logs; log failures in defect tracker.

Important Note:

Using Agile principle, these testing will take place iteratively over the period of development, not limited to at the end.

© UMHackathon 2026

Email: umhackathon@um.edu.my